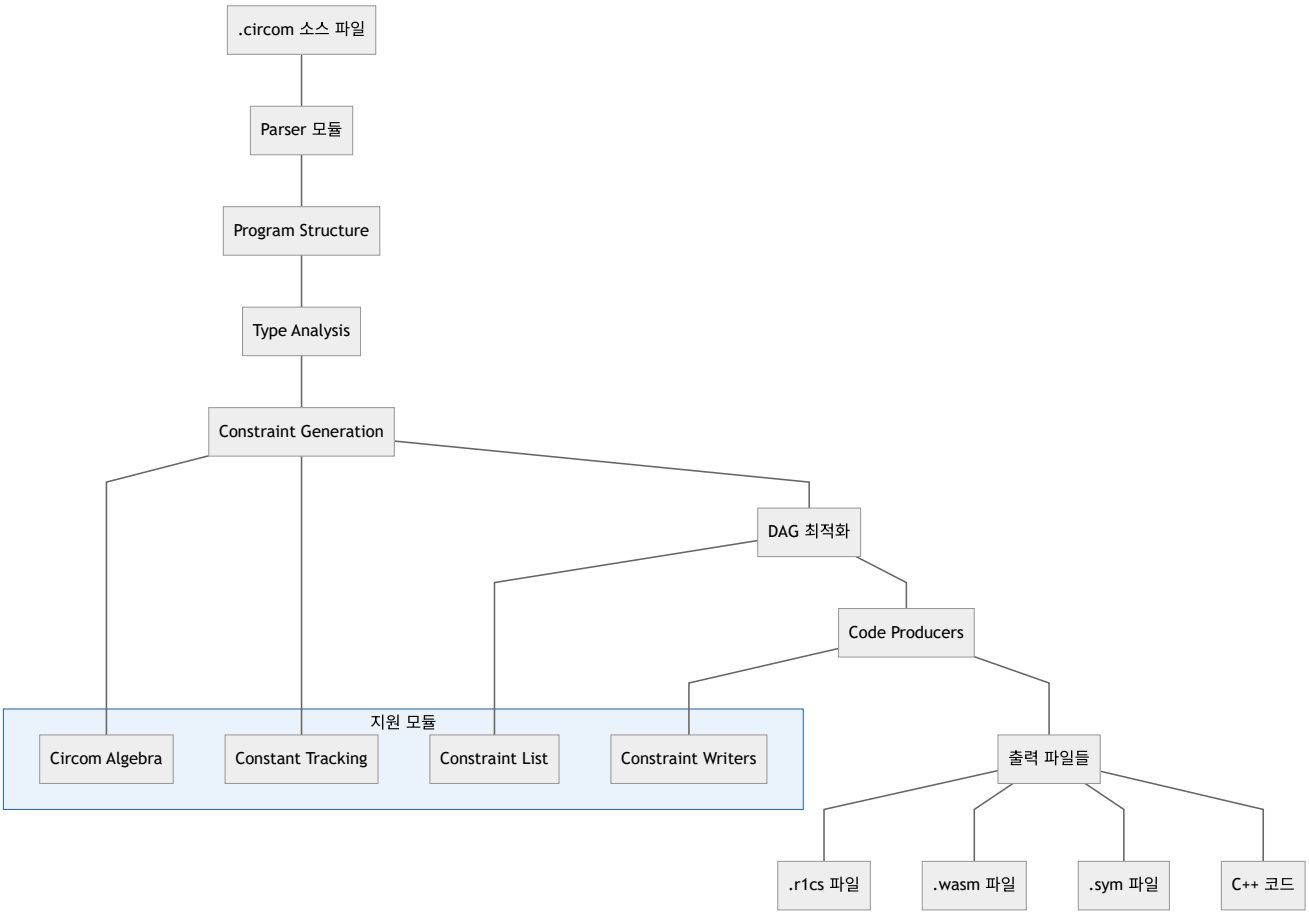
Circom 컴파일러 전체 아키텍처 문서

개요

Circom 컴파일러는 zkSNARK 증명 시스템을 위한 산술 회로를 정의하는 도메인 특화 언어(DSL)의 컴파일러입니다. 이 문서는 Circom 컴파일러의 전체 아키텍처, 12개 주요 모듈의 역할과 상호작용, 그리고 전체 컴파일 파이프라인을 상세히 설명합니다.

전체 컴파일 파이프라인



컴파일 과정 요약

1. **파싱 단계**: `.circom` 파일을 AST(추상 구문 트리)로 변환

2. **타입 분석 단계**: 신호 타입 검증 및 템플릿 매개변수 추론

3. **제약 생성 단계**: 회로 실행 및 R1CS 제약 조건 생성

4. **최적화 단계**: DAG 기반 제약 조건 최적화

5. **코드 생성 단계**: 다양한 형태의 출력 파일 생성

12개 주요 모듈 상세 분석

1. Parser 모듈

역할: LALRPOP 기반 문법 파싱 및 AST 생성

핵심 함수: `run_parser`

의존성: `program_structure`, `lalrpop-util`, `regex`

```
pub fn run_parser(
initial_file: String,
version: &str,
link_libraries: Vec<PathBuf>,
prime: &BigInt,
flag_no_init: bool
) -> Result<(ProgramArchive, ReportCollection), (FileLibrary, ReportCollection)>
```

주요 기능:

- `.circom` 파일의 토큰화 및 구문 분석
- Include 파일 처리 및 의존성 그래프 구축
- 문법 설탕(Syntactic Sugar) 제거

- 버전 호환성 검사

2. Program Structure 모듈

역할: AST 노드 정의 및 프로그램 구조 관리

핵심 데이터 구조: `Template`, `Component`, `Signal`, `ProgramArchive`

의존성: `codespan`, `num-bigint-dig`, `serde`

```
pub struct Template {
pub name: String,
pub args: Vec<String>,
pub body: Statement,
pub parallel: bool,
pub custom_gates: bool,
}

pub struct ProgramArchive {
pub file_library: FileLibrary,
pub main_component: Option<String>,
pub templates: HashMap<String, Template>,
}
```

주요 기능:

- AST 노드 구조 정의
- 프로그램 메타데이터 관리

- 에러 리포팅 시스템
- 파일 라이브러리 관리

3. Type Analysis 모듈

역할: 타입 체킹 및 의미 분석

핵심 함수: `analyse_project`

의존성: `program_structure`

```
pub fn analyse_project(program_archive: &mut ProgramArchive) -> Result<(), ()>
```

주요 기능:

- 신호(Signal) 타입 검증 (Input, Output, Intermediate)
- 템플릿 매개변수 타입 추론
- 컴포넌트 인스턴스화 검증
- 회로 연결성 분석

4. Constraint Generation 모듈

역할: 제약 조건 생성 및 회로 실행

핵심 함수: `build_circuit`

의존성: `compiler`, `circom_algebra`, `program_structure`, `constraint_writers`, `constraint_list`, `dag`

```
pub fn build_circuit(
program_archive: ProgramArchive,
config: BuildConfig
```

```
) -> Result<(Box<dyn ConstraintExporter>, VCP), ()>
```

주요 기능:

- 템플릿 인스턴스화 (`instantiation`)
- R1CS 제약 조건 생성 (`constraint_execution`)
- VCP(Very Concrete Program) 생성 (`export`)
- 회로 실행 및 증인(Witness) 생성

5. Circom Algebra 모듈

역할: 필드 연산 및 산술 표현식 처리

핵심 데이터 구조: `ArithmeticExpression` , `ArithmeticError`

의존성: `constant_tracking` , `num-bigint-dig`

주요 기능:

- 유한체(Finite Field) 연산 구현
- 다항식 연산 및 계수 처리
- 산술 표현식 최적화
- 상수 전파 지원

6. Constant Tracking 모듈

역할: 상수 추적 및 최적화

의존성: 없음 (기본 모듈)

주요 기능:

- 컴파일 타임 상수 계산
- 상수 전파 알고리즘

- 동적 값과 정적 값 구분
- 최적화를 위한 상수 정보 제공

7. DAG 모듈

역할: 의존성 그래프 및 최적화

핵심 함수: `map_to_list`, `produce_witness`

의존성: `constraint_list`, `constraint_writers`, `circom_algebra`, `program_structure`

```
impl DAG {
pub fn map_to_list(dag: DAG, flags: SimplificationFlags) -> ConstraintList
pub fn produce_witness(dag: &DAG) -> Vec<BigInt>
}
```

주요 기능:

- 제약 조건 의존성 그래프 구축
- 병렬 처리 최적화
- 죽은 코드 제거 (Dead Code Elimination)
- 제약 조건 병합 및 단순화

8. Constraint List 모듈

역할: 제약 조건 리스트 관리

의존성: `constraint_writers`, `circom_algebra`, `program_structure`

주요 기능:

- R1CS 제약 조건 리스트 관리

- 병렬 처리 지원 (`threadpool`)
- JSON 형태 제약 조건 출력
- 제약 조건 검증 및 일관성 체크

9. Constraint Writers 모듈

역할: R1CS 파일 생성 및 출력

핵심 인터페이스: `ConstraintExporter`

의존성: `circom_algebra`

```
pub trait ConstraintExporter {
fn r1cs(&self, file: &str, custom_gates: bool) -> Result<(), ()>;
fn sym(&self, file: &str) -> Result<(), ()>;
fn json_constraints(&self, debug: &DebugWriter) -> Result<(), ()>;
}
```

주요 기능:

- .r1cs 바이너리 파일 생성
- .sym 심볼 파일 생성
- JSON 형태 제약 조건 출력
- 디버깅 정보 생성

10. Code Producers 모듈

역할: C++ 및 WebAssembly 코드 생성

의존성: `circom_algebra` , `handlebars` , `serde_json`

주요 기능:

- C++ 증명 생성 코드 생성 (`produce_c_code`)
- WebAssembly 코드 생성 (`produce_wasm_code`)
- 템플릿 기반 코드 생성 (Handlebars)
- snarkjs/rapidSnark 연동 코드 생성

11. Compiler 모듈

역할: 중간 표현(IR) 및 컴파일러 인터페이스

의존성: `constant_tracking` , `program_structure` , `code_producers`

주요 기능:

- HIR(High-level Intermediate Representation) 관리
- VCP(Very Concrete Program) 데이터 구조
- 컴파일러 설정 관리
- 코드 생성 인터페이스 제공

12. Circom 모듈 (메인 실행 엔진)

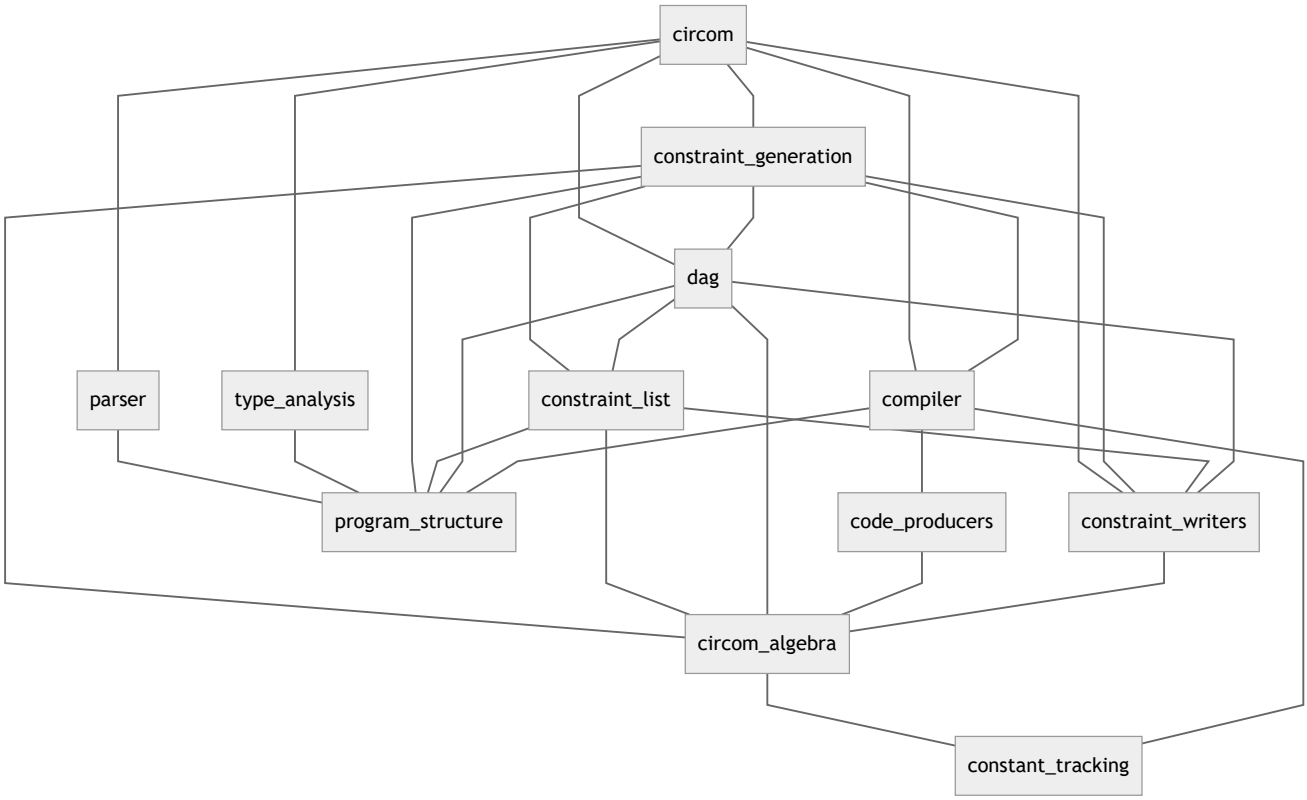
역할: 메인 실행 엔진 및 사용자 인터페이스

의존성: 모든 주요 모듈들

주요 기능:

- 명령행 인터페이스 (CLI) 제공
- 전체 컴파일 파이프라인 조정
- 사용자 입력 처리 및 설정 관리
- 에러 리포팅 및 출력 관리

모듈 간 의존성 그래프



의존성 계층 구조

1. **기반 계층**: `constant_tracking`, `program_structure`
2. **연산 계층**: `circom_algebra`
3. **분석 계층**: `parser`, `type_analysis`
4. **실행 계층**: `constraint_generation`, `dag`
5. **출력 계층**: `constraint_writers`, `constraint_list`, `code_producers`

6. **통합 계층**: `compiler`

7. **인터페이스 계층**: `circom`

주요 데이터 흐름

1. 파싱 단계

```
.circom 파일 → LALRPOP 파서 → AST → ProgramArchive
```

2. 타입 분석 단계

```
ProgramArchive → 타입 체커 → 검증된 ProgramArchive
```

3. 제약 생성 단계

```
ProgramArchive → 템플릿 인스턴스화 → ExecutedProgram → DAG + VCP
```

4. 최적화 단계

```
DAG → 단순화 알고리즘 → 최적화된 ConstraintList
```

5. 출력 생성 단계

```
ConstraintList + VCP → 다양한 Exporter → .r1cs, .wasm, .sym, C++ 파일
```

성능 최적화 특징

병렬 처리

- `constraint_list` 모듈의 `threadpool` 활용
- DAG 기반 병렬 제약 조건 처리
- 멀티코어 CPU 활용 (`num_cpus`)

메모리 최적화

- 지연 평가 (Lazy Evaluation)
- 스트리밍 파싱
- 효율적인 BigInt 연산 (`num-bigint-dig`)

컴파일 시간 최적화

- 증분 컴파일 지원
- 상수 전파 및 죽은 코드 제거
- 효율적인 의존성 그래프 관리

확장성 및 모듈성

플러그인 아키텍처

- `ConstraintExporter` 트레이트를 통한 출력 형식 확장
- 템플릿 기반 코드 생성 (Handlebars)
- 모듈별 독립적인 테스트 및 개발

언어 지원

- 다국어 에러 메시지 지원
- 컬러 출력 (`ansi_term`)
- 상세한 소스 위치 추적 (`codespan`)

이 아키텍처는 Circom 컴파일러의 복잡한 요구사항을 효율적으로 처리하면서도 확장 가능하고 유지보수가 용이한 구조를 제공합니다.

상세

의존성 분석

각 모듈별 Cargo.toml 의존성 분석

1. Parser 모듈

```
[dependencies]

program_structure = {path = "../program_structure"}

lalrpop-util = { version="0.19.9", features = ["lexer"]}

regex = "1.1.2"

rustc-hex = "2.0.1"

num-bigint-dig = "0.8.4"

num-traits = "0.2.6"

serde = "1.0.82"

serde_derive = "1.0.91"


[build-dependencies]
```

```
lalrpop = { version = "0.19.9", features = ["lexer"] }
```

- **내부 의존성**: `program_structure`
- **외부 의존성**: LALRPOP 파서 생성기, 정규표현식, 직렬화
- **빌드 의존성**: LALRPOP 컴파일러

2. Program Structure 모듈

```
[dependencies]

codespan = "0.9.0"

codespan-reporting = "0.9.0"

regex = "1.1.2"

rustc-hex = "2.0.1"

num-bigint-dig = "0.8.4"

num-traits = "0.2.6"

serde = "1.0.82"

serde_derive = "1.0.91"
```

- **내부 의존성**: 없음 (기반 모듈)
- **외부 의존성**: 소스 위치 추적, 에러 리포팅, 수치 연산

3. Type Analysis 모듈

```
[dependencies]

program_structure = {path = "../program_structure"}

num-bigint-dig = "0.8.4"
```

```
num-traits = "0.2.6"
```

- **내부 의존성**: `program_structure`
- **외부 의존성**: 수치 연산 라이브러리

4. Circom Algebra 모듈

```
[dependencies]
num-bigint-dig = "0.8.4"
num-traits = "0.2.6"
constant_tracking = {path = "../constant_tracking"}
```

- **내부 의존성**: `constant_tracking`
- **외부 의존성**: 수치 연산 라이브러리

5. Constant Tracking 모듈

```
[dependencies]
# 의존성 없음
```

- **내부 의존성**: 없음
- **외부 의존성**: 없음 (순수 Rust 구현)

6. Constraint Generation 모듈

```
[dependencies]
ansi_term = "0.12.1"
```

```
num-traits = "0.2.6"

num-bigint-dig = "0.8.4"

compiler = { path = "../compiler" }

circom_algebra = { path = "../circom_algebra" }

program_structure = { path = "../program_structure" }

constraint_writers = { path = "../constraint_writers" }

constraint_list = { path = "../constraint_list" }

dag = { path = "../dag" }
```

- **내부 의존성**: `compiler`, `circom_algebra`, `program_structure`, `constraint_writers`, `constraint_list`, `dag`
- **외부 의존성**: 컬러 터미널 출력, 수치 연산

7. DAG 모듈

```
[dependencies]

constraint_list = { path = "../constraint_list" }

constraint_writers = { path = "../constraint_writers" }

circom_algebra = { path = "../circom_algebra" }

program_structure = { path = "../program_structure" }

json = "0.12.4"
```

- **내부 의존성**: `constraint_list`, `constraint_writers`, `circom_algebra`, `program_structure`
- **외부 의존성**: JSON 처리

8. Constraint List 모듈

```
[dependencies]

constraint_writers = { path = "../constraint_writers" }

circom_algebra = { path = "../circom_algebra" }

program_structure = { path = "../program_structure" }

json = "0.12.4"

threadpool = "1.0"

num_cpus = "1.0"
```

- **내부 의존성**: `constraint_writers`, `circom_algebra`, `program_structure`
- **외부 의존성**: JSON 처리, 스레드 풀, CPU 코어 수 감지

9. Constraint Writers 모듈

```
[dependencies]

circom_algebra = { path = "../circom_algebra" }

json = "0.12.4"
```

- **내부 의존성**: `circom_algebra`
- **외부 의존성**: JSON 처리

10. Code Producers 모듈

```
[dependencies]

circom_algebra = { path = "../circom_algebra" }
```

```
handlebars = "4.1.3"

lz_fnv = "0.1.2"

num-bigint-dig = "0.8.4"

serde_json = "1.0.68"
```

- **내부 의존성**: `circom_algebra`
- **외부 의존성**: 템플릿 엔진, 해시 함수, JSON 직렬화

11. Compiler 모듈

```
[dependencies]

constant_tracking = {path = "../constant_tracking"}

program_structure = {path = "../program_structure"}

code_producers = {path = "../code_producers"}

num-bigint-dig = "0.8.4"

num-traits = "0.2.6"
```

- **내부 의존성**: `constant_tracking` , `program_structure` , `code_producers`
- **외부 의존성**: 수치 연산

12. Circom 모듈 (메인)

```
[dependencies]

parser = {path = "../parser"}

program_structure = { path = "../program_structure" }

type_analysis = { path = "../type_analysis" }
```

```
constraint_generation = { path = "../constraint_generation" }

constraint_writers = { path = "../constraint_writers" }

compiler = { path = "../compiler" }

dag = { path = "../dag" }

clap = "2.33.0"

ansi_term = "0.12.1"

wast = "39.0.0"

exitcode = "1.1.2"
```

- **내부 의존성**: 거의 모든 주요 모듈
- **외부 의존성**: CLI 파싱, 터미널 출력, WebAssembly 텍스트 처리

의존성 그래프 분석

순환 의존성 검사

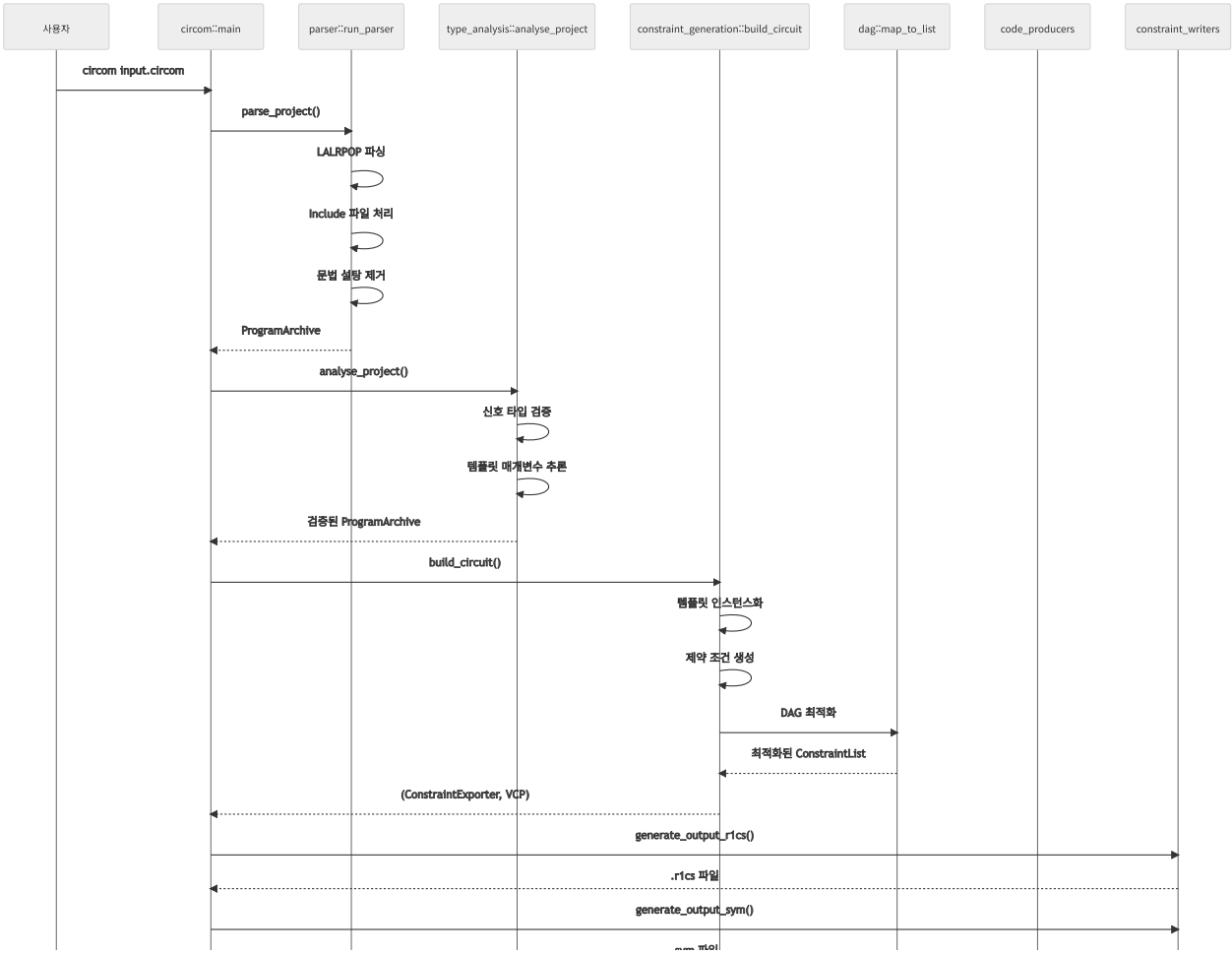
현재 아키텍처는 순환 의존성이 없는 DAG(Directed Acyclic Graph) 구조를 유지합니다:

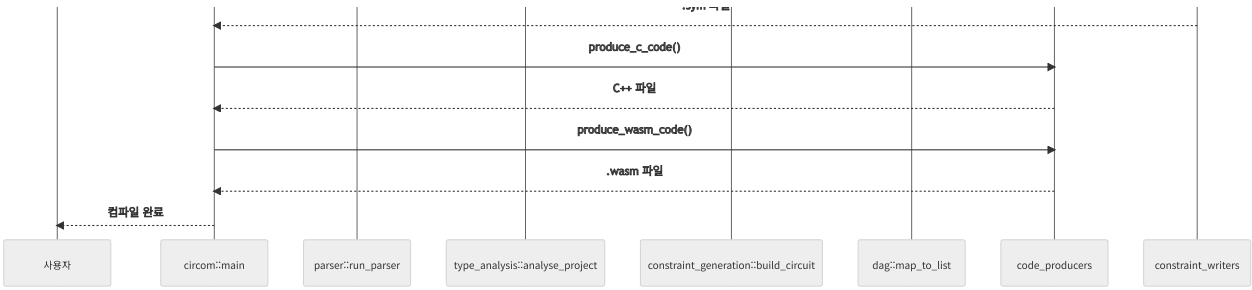
1. **Leaf 노드**: `constant_tracking`, `program_structure`
2. **Level 1**: `circom_algebra`, `type_analysis`, `parser`
3. **Level 2**: `constraint_writers`, `code_producers`
4. **Level 3**: `constraint_list`, `compiler`
5. **Level 4**: `dag`
6. **Level 5**: `constraint_generation`
7. **Root**: `circom`

핵심 의존성 패턴

1. `program_structure` : 거의 모든 모듈이 의존하는 기반 모듈
2. `circom_algebra` : 수치 연산이 필요한 모든 모듈이 의존
3. `constraint_writers` : 출력 관련 모듈들이 공통으로 의존
4. `num-bigint-dig` : 대부분의 모듈에서 사용하는 큰 정수 연산

컴파일 파이프라인 상세 플로우차트


사용자
circom::main
parser::run_parser
type_analysis::analyse_project
constraint_generation::build_circuit
dag::map_to_list
code_producers
constraint_writers
circom input.circom
parse_project()
LALRPOP 파싱
Include 파일 처리
문법 설탕 제거
ProgramArchive
analyse_project()
신호 타입 검증
템플릿 매개변수 추론
검증된 ProgramArchive
build_circuit()
템플릿 인스턴스화
제약 조건 생성
DAG 최적화
최적화된 ConstraintList
(ConstraintExporter, VCP)
generate_output_r1cs()
.r1cs 파일
generate_output_sym()



이 상세한 분석을 통해 Circom 컴파일러의 모듈 구조와 의존성 관계를 완전히 이해할 수 있으며, 각 단계에서의 데이터 변환과 처리 과정을 명확히 파악할 수 있습니다.